



Rapport de Sprint

Advanced Drupal Development



*Data & Hooks avancés
Entity, Field, Configuration API & Extension
Mechanisms*

Réalisé par

Hamza Lazaar

Encadré par

**Abdelmajid Bendrif
Hamza Bahlaouane**

Table des matières

1 Jour 1 : Work with Data (Entity & Field API, Configuration Entities)	4
1.1 Altération d'Entités et Contraintes	4
1.1.1 Ajout d'une Contrainte de Mot de Passe	4
1.2 Système d'Accès (Access API)	6
1.2.1 Comprendre AccessResult	6
1.3 Création d'Entités Custom	6
1.3.1 Génération via Drush	6
1.4 Manipulations de l'API Field et Configuration	7
1.4.1 Récupération de la définition d'un champ en PHP	7
1.4.2 Multiples Formatters pour un Type de Champ	7
1.4.3 Gérer la configuration via Drush	8
2 Jour 2 : Work with Hooks	9
2.1 Création de champs via hook	9
2.1.1 hook_entity_base_field_info	9
2.2 Le Cycle de Vie d'un Module (Install & Updates)	10
2.2.1 Role de hook_install	10
2.2.2 Role de hook_update_N	10
2.3 Interventions sur la sauvegarde (<i>Presave hook</i>)	11
2.3.1 Préfixer un titre avec The hook_ENTITY_TYPE_presave	11
2.3.2 Le rôle de l'objet <code>\$entity->original</code>	12
2.4 Système de Templates (Theme Hooks)	12
2.4.1 Surcharge d'un Theme Hook tiers	12
2.4.2 Créer des Theme Suggestions dynamiquement	12
3 Jour 3 : Work with configuration & features	14
3.1 Mise en place des environnements	14
3.1.1 Configuration du serveur local (Apache & MySQL)	14
3.2 Synchronisation de Configuration	14
3.2.1 Obstacles lors de l'import (Drush config :import)	16
3.3 Résolution de l'UUID et Stockages de configuration	17
3.3.1 Active Storage vs Sync Storage	17
3.3.2 Pourquoi drush cget + cset a échoué initialement ?	17
3.3.3 La solution	17
3.4 Gestion de Configuration avec Features	18
3.4.1 Installation et Dépendances	18
3.4.2 Création et Déploiement d'une Feature	18
3.4.3 Gestion des Conflits (<i>Overrides</i>)	20
4 Jour 4 : Work with Cache	22

4.1	Mise en place d'un API endpoint	22
4.1.1	Création de la route	22
4.2	Le Cache basé sur le Temps (Max-Age)	22
4.3	Le Cache basé sur les Entités (Cache-Tags)	23
4.4	Inspection des Cache Tags	23
5	Jour 5 : Work with Data Migration	25
5.1	Le rôle du plugin <code>migration_lookup</code>	25
5.2	Importer depuis une base de données MySQL	25
5.3	Annulation (Rollback) d'une migration	26
5.4	Traitement des données de champ (Pipeline)	26

| Chapitre 1- Jour 1 : Work with Data (Entity & Field API, Configuration Entities)

Dans ce premier chapitre, nous plongeons au cœur de l'architecture des données de Drupal, en explorant les Entités, l'API des champs (Field API), ainsi que les méthodes pour manipuler la configuration et intercepter les processus de validation de données.

1.1. Altération d'Entités et Contraintes

1.1.1. Ajout d'une Contrainte de Mot de Passe

Pour étendre la sécurité lors de la création ou modification des utilisateurs, nous pouvons utiliser l'*alter hook* `hook_entity_base_field_info_alter()`. Ce dernier nous permet d'interagir avec les définitions de champs d'une entité existante.

L'objectif était d'ajouter une contrainte au champ `pass` de l'entité `user`, en utilisant le service de validation du module contrib *Password Policy*.

```
1  /**
2   * Implements hook_entity_base_field_info_alter().
3   */
4   function my_module_entity_base_field_info_alter(&$fields, \Drupal\
5     Core\Entity\EntityTypeInterface $entity_type) {
6     // Alter the password field on the user entity.
7     if ($entity_type->id() === 'user' && isset($fields['pass'])) {
8         $fields['pass']->addConstraint('PasswordPolicy');
9     }
10 }
```

Listing 1.1 – Implémentation de `hook_entity_base_field_info_alter`

Pour que cette contrainte fonctionne, nous avons déclaré le plugin `PasswordPolicyConstraint` et son validateur associé `PasswordPolicyConstraintValidator` dans le dossier `src/Plugin/Validation/Constraint`. Le validateur utilise l'injection de dépendances (`ContainerInjectionInterface`) pour récupérer le service `password_policy.validator`.

```
1  use Drupal\Core\DependencyInjection\ContainerInjectionInterface;
2  use Symfony\Component\DependencyInjection\ContainerInterface;
3  use Symfony\Component\Validator\ConstraintValidator;
4
5  class PasswordPolicyConstraintValidator extends ConstraintValidator
6    implements ContainerInjectionInterface {
7      protected $passwordPolicyValidator;
```

```

7
8 public function __construct($password_policy_validator) {
9     if ($password_policy_validator) {
10         $this->passwordPolicyValidator = $password_policy_validator;
11     }
12 }
13
14 public static function create(ContainerInterface $container) {
15     $validator = NULL;
16     if ($container->has('password_policy.validator')) {
17         $validator = $container->get('password_policy.validator');
18     }
19     return new static($validator);
20 }
21 }

```

Listing 1.2 – Injection du service dans le ConstraintValidator

```

15 class PasswordPolicyConstraintValidator extends ConstraintValidator implements ContainerInjectionInterface
51
52 /**
53  * {@inheritdoc}
54  */
55 @hamza-lzr *
56 public function validate($items, Constraint $constraint)
57 {
58     if (!$this->passwordPolicyValidator) {
59         // If the service is not available, do not block validation.
60         return;
61     }
62     $password = $items->value;
63
64     // Typically, passwords could be empty when a user is just submitting a register request
65     // We skip validation if it's empty.
66     if (empty($password)) {
67         return;
68     }
69
70     if (strlen($password) < 8) {
71         $constraint->message = "Password length must be higher than 8";
72         $this->context->addViolation($constraint->message);
73     }
74
75     $user = \Drupal::currentUser();
76
77     $entity = $this->context->getRoot()->getValue();
78     if ($entity instanceof \Drupal\user\UserInterface) {
79         $user = $entity;
80     }
81     else {
82         $user = \Drupal\user\Entity\User::load($user->id());
83     }
84
85     $validation_report = $this->passwordPolicyValidator->validatePassword($password, $user);
86
87     if ($validation_report->isValid()) {
88         $this->context->addViolation($validation_report->getErrors()->render());
89     }
90 }

```

\\Drupal\\content_entity_example\\Plugin\\Validation\\Constraint\\ PasswordPolicyConstraintValidator

FIGURE 1.1 – Le fichier PasswordPolicyConstraintValidator.php validant le mot de passe via l'API

Error message

Password length must be higher than 8

Email address *

email@drup.al

The email address is not made public. It will only be used if you need to be contacted about your account or for opted-in notifications.

Username *

email

Several special characters are allowed, including space, period (.), hyphen (-), apostrophe ('), underscore (_), and the @ sign.

Password

To change the current user password, enter the new password in both fields.

FIGURE 1.2 – Test d’un mot de passe avec moins de 8 caractères

1.2. Système d’Accès (Access API)

1.2.1. Comprendre AccessResult

Dans Drupal, la gestion des accès ne renvoie pas un simple booléen (vrai/faux). Drupal utilise des objets de type **AccessResult** pour déterminer si un utilisateur a le droit d’effectuer une action (voir, modifier, supprimer) sur une entité ou une route.

Les trois réponses possibles sont :

- **AccessResult::allowed()** : Autorise explicitement l’accès.
- **AccessResult::forbidden()** : Interdit explicitement l’accès (écrase toute permission).
- **AccessResult::neutral()** : Ni autorise, ni interdit (laisse les autres modules décider).

De plus, ces objets peuvent transporter des métadonnées de cache (**CacheabilityMetadata**). Ainsi, si l’accès est calculé en fonction du rôle ”premium”, on peut lier cet **AccessResult** au **Cache Context** `user.roles`, et Drupal mettra l’accès en cache de façon optimale.

1.3. Création d’Entités Custom

1.3.1. Génération via Drush

La création d’une nouvelle **Content Entity** implique beaucoup de code répétitif (interfaces, formulaires, routes, handlers). Pour gagner un temps précieux, on ne les crée presque jamais à la main.

La méthode recommandée est d’utiliser le générateur Drush :

```
drush generate entity:content
```

Listing 1.3 – Génération d’une entité via Drush

Cet outil interactif pose une série de questions (nom de l'entité, module cible, prise en charge des *Revisions*, *Translations*, etc.) et génère instantanément toute la structure de fichiers requise pour disposer d'une entité pleinement fonctionnelle.



```
hamza@Hamzas-MacBook-Pro ~/dev/new_drupal master ++ vendor/bin/drush generate entity:content

Welcome to content-entity generator!
-----

Module machine name:
> mock_entity

Module name [Mock entity]:
>

Entity type label [Mock entity]:
>

Entity type ID [mock_entity]:
>

Entity class [MockEntity]:
>

Entity base path [/mock-entity]:
>
```

FIGURE 1.3 – Génération interactive d'une entité via Drush

1.4. Manipulations de l'API Field et Configuration

1.4.1. Récupération de la définition d'un champ en PHP

Pour récupérer une définition de champ depuis le code, il convient d'interroger l'`EntityFieldManager`.

```
1 $entity_field_manager = \Drupal::service('entity_field.manager');
2 // Recupere tous les Base Fields d'une entite :
3 $base_fields = $entity_field_manager->getBaseFieldDefinitions('node'
4 );
5
6 // Recupere les Bundle Fields specifiques d'un type :
7 $bundle_fields = $entity_field_manager->getFieldDefinitions('node',
8 'article');
9 $mon_champ = $bundle_fields['field_custom_data'];
```

Listing 1.4 – Récupérer la définition d'un champ via le conteneur

1.4.2. Multiples Formatters pour un Type de Champ

Oui, il est totalement possible (et meme courant) de creer plusieurs *Field Formatters* pour un meme *Field Type*. Par exemple, un champ Image (Field Type) peut avoir un formatter pour afficher l'image brute, un autre pour l'afficher en tant qu'URL, ou encore sous forme de galerie.

Pour cela, on crée de nouvelles classes de plugin étendues de `FormatterBase` (dans le répertoire `src/Plugin/Field/FieldFormatter`), et on s'assure que l'annotation stipule le même type de champ ciblé :

```
1 /**
2  * Plugin annotation example.
3  *
4  * @FieldFormatter(
5  *   id = "my_custom_image_gallery",
```

```
6  *    label = @Translation("My Gallery Formatter"),
7  *    field_types = {
8  *        "image"
9  *    }
10 * )
11 */
```

Listing 1.5 – Annotation d'un nouveau formatter

1.4.3. Gérer la configuration via Drush

Afin de lire ou retrouver un paramètre de configuration propre à un module depuis le terminal, Drush propose les commandes `config:get` (ou `cget`).

```
1  # Affiche toute la configuration d'un module
2  drush config:get my_module.settings
3
4  # Affiche une clef ou valeur precise
5  drush config:get my_module.settings api_key
6
7  # Editer la configuration
8  drush config:edit my_module.settings
```

Listing 1.6 – Récupérer et éditer une config via Drush

Chapitre 2 - Jour 2 : Work with Hooks

Le deuxième jour est consacré aux "Hooks", le système fondamental d'extension événementielle dans Drupal, qui a forgé sa philosophie de développement depuis ses toutes premières versions.

2.1. Création de champs via hook

2.1.1. hook_entity_base_field_info

Si l'on cherche à ajouter un **Base Field** à une entité tierce procéduralement, on peut le déclarer via l'*alter hook* `hook_entity_base_field_info`.

Par exemple, pour insérer un champ à notre Custom Entity *News* depuis un autre module, et l'exposer dans les *View Modes* (Manage Display) et *Form Modes* (Manage form display) :

```
1  /**
2   * Implements hook_entity_base_field_info().
3   */
4  function content_entity_example_entity_base_field_info(
5      EntityTypeInterface $entity_type) {
6      $fields = [];
7      if ($entity_type->id() === 'news') {
8          $fields['custom_news_data'] = BaseFieldDefinition::create('
9              string')
10             ->setLabel(t('Custom Data'))
11             ->setDisplayOptions('form', [
12                 'type' => 'string_textfield',
13                 'weight' => 5,
14             ])
15             ->setDisplayConfigurable('form', TRUE)
16             ->setDisplayConfigurable('view', TRUE);
17     }
18     return $fields;
19 }
```

Listing 2.1 – Définition procédurale d'un champ base

Home > Administration > Structure > News

Manage display ☆

Settings Manage fields Manage form display **Manage display**

+ Add field group

Field	Machine name	Label	Format	
÷ Label	label	- Hidden -	Plain text	
÷ Status	status	Above	Boolean	Display: Enabled / Disabled
÷ Custom Data	custom_news_data	Above	Plain text	
÷ Author	uid	Above	Author	
÷ Authored on	created	Above	Default	Date format: medium Tooltip date format: long

FIGURE 2.1 – Le champ "Custom Data" apparaît sur le formulaire d'édition de la News

2.2. Le Cycle de Vie d'un Module (Install & Updates)

Les fichiers `.install` gèrent le cycle de vie du module et la structure de sa base de données lors de son activation et de ses montées en version.

2.2.1. Role de `hook_install`

Ce hook est déclenché de façon unique, **au moment exact où un module est activé pour la toute première fois**.

Il sert à exécuter de la logique PHP qui ne peut pas être externalisée via de simples fichiers de configuration YAML. Par exemple, calculer un chemin d'accès absolu au serveur et le sauvegarder, créer une taxonomie initiale, créer de faux nœuds d'accueil pour le module, ou assigner des droits dynamiquement à un rôle existant.

2.2.2. Role de `hook_update_N`

Contrairement au précédent, le hook `hook_update_N` est prévu pour interagir sur des instances de modules **déjà installés**. Il permet la migration de données ou la modification de schémas de base de données en direct via `drush updb`.

Même pour le champ ajouté via `hook_entity_base_field_info` à l'entité News, Drupal doit être informé s'il s'agit d'une instance existante :

```

1  /**
2   * Add the 'custom_news_data' base field to the 'news' entity type.
3   */
4  function content_entity_example_update_9001() {
5      $manager = \Drupal::entityDefinitionUpdateManager();
6      $entity_type = $manager->getEntityType('news');
7
8      // Installe le schema de l'entite s'il manquait
9      $change_list = $manager->getChangeList();
10     if (!empty($change_list['news']['entity_type'])) {

```

```

11     $manager->installEntityType(\Drupal::entityTypeManager()->
12         getDefinition('news'));
13
14     $field_storage = BaseFieldDefinition::create('string')
15         ->setLabel(t('Custom Data'));
16
17     // Ajoute la colonne dans la base de donnees
18     if (!$manager->getFieldStorageDefinition('custom_news_data', '
19         news')) {
20         $manager->installFieldStorageDefinition('custom_news_data',
21             'news', 'content_entity_example', $field_storage);
22     }
23 }

```

Listing 2.2 – Exemple de hook update

2.3. Interventions sur la sauvegarde (*Presave hook*)

2.3.1. Préfixer un titre avec The hook_ENTITY_TYPE_presave

Afin d'intercepter une entité juste avant qu'elle ne soit sauvegardée dans la base de données, le *presave hook* intervient. En ciblant l'**Entity Type** Node, nous vérifions si l'entité `isNew()` (n'est pas une simple mise à jour), et modifions son titre dynamiquement.

```

1  /**
2   * Implements hook_ENTITY_TYPE_presave() for 'node' entities.
3   */
4  function content_entity_example_node_presave(\Drupal\Core\Entity\
5      EntityInterface $entity): void {
6      if ($entity->isNew() && $entity->bundle() === 'article') {
7          $current_title = $entity->getTitle();
8          if (strpos($current_title, 'HEY-') !== 0) {
9              $entity->setTitle('HEY-' . $current_title);
10         }
11     }
12 }

```

Listing 2.3 – Le presave hook pour préfixer les nouveaux articles

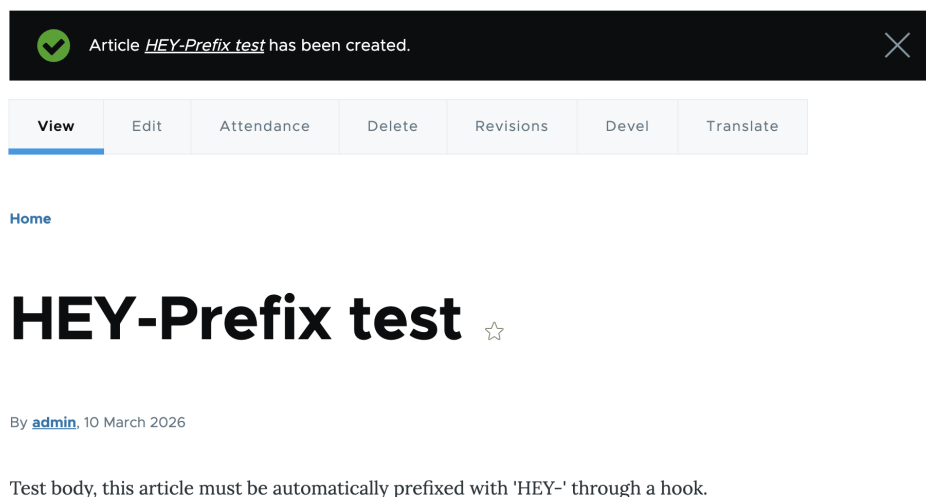


FIGURE 2.2 – Le titre désormais préfixé par HEY-

2.3.2. Le rôle de l'objet `$entity->original`

Lorsqu'une **mise à jour** est effectuée sur une entité (exemple lors du `hook_node_update`), Drupal charge au préalable l'état exact de l'entité avant la modification et le place dans `$entity->original`.

Cela permet aux développeurs de faire des comparaisons ("*Est-ce que le statut est passé de Fermé à Ouvert ?*"). Si un utilisateur avait remplacé une image en uploadant une nouvelle, l'original permet d'isoler l'ancienne ID et de la supprimer du serveur sans affecter la nouvelle image. S'il s'agit d'une création (nouvelle entité), `original` vaut `NULL`.

2.4. Système de Templates (Theme Hooks)

La surécriture du front-end et des conventions esthétiques passe par le Theme Registry.

2.4.1. Surcharge d'un Theme Hook tiers

Pour surcharger un template fourni par un autre module :

- **Depuis un theme** : Activer le Twig Debugging, vérifier le nom suggéré du template, et recréer le fichier avec ce nom dans le répertoire `templates/` de son thème. Drupal interceptera ce template par défaut.
- **Depuis un module Custom** : Implémenter `hook_theme_registry_alter()` pour pointer le chemin vers son module et modifier l'adresse de surcharge interne.
- **Pour les variables uniquement** : Utiliser simplement `hook_preprocess_HOOK` (comme `hook_preprocess_node`) pour n'ajouter que des logiques PHP (nouvelles classes CSS, nouveaux paramètres) tout en conservant le fichier *Twig* initial.

2.4.2. Créer des Theme Suggestions dynamiquement

Lorsque la résolution du *View Mode* de l'entité "User" pose des contraintes (parce que Drupal n'offre par défaut que le template générique `user.html.twig`), l'*alter hook* `hook_theme_suggestions_alter()` permet de générer des **Theme Suggestions** spécifiques (comme `user--compact.html.twig`).

```
1 /**
2  * Implements hook_theme_suggestions_alter().
3  */
4 function content_entity_example_theme_suggestions_alter(array &
5   $suggestions, array $variables, $hook) {
6   if ($hook === 'user') {
7     if (isset($variables['elements']['#view_mode'])) {
8       $view_mode = $variables['elements']['#view_mode'];
9       $sanitized_view_mode = strtr($view_mode, '-', '_');
10
11       $suggestions[] = 'user_' . $sanitized_view_mode;
12     }
13   }
14 }
```

Listing 2.4 – Ajout d’une suggestion de mode d’affichage pour les utilisateurs

| Chapitre 3- Jour 3 : Work with configuration & features

Ce troisième jour a été consacré à la compréhension du système de **Configuration Management (CMI)** de Drupal et à la manière de synchroniser des données de structure entre différents environnements (développement et production).

3.1. Mise en place des environnements

Afin de simuler un flux de travail réaliste, nous avons déployé deux instances Drupal 11 complètement indépendantes en local : `drupal-dev` et `drupal-prod`.

3.1.1. Configuration du serveur local (Apache & MySQL)

- **Serveur Web (Apache)** : Nous avons configuré des *Virtual Hosts* Apache natifs sur macOS pour pointer vers les dossiers `web/` de chaque instance (ex : `drupal-dev.local`). Un défi technique rencontré fut le conflit de port, car Homebrew Apache écoute par défaut sur le port 8080, nécessitant une modification du `httpd.conf` vers le port 80 (et l'utilisation de `sudo` pour le redémarrage).
- **Base de données (MySQL)** : Les bases de données `drupal_dev` et `drupal_prod` ont été provisionnées au sein d'un conteneur Docker (OrbStack) exposé sur le port 3307. Lors de l'installation via Drush, l'URL de connexion a dû être explicite quant au port (ex : `mysql://root:root@127.0.0.1:3307/drupal_dev`).

3.2. Synchronisation de Configuration

Sur l'environnement de développement, nous avons modélisé une base :

- Installation du module `devel`.
- Création d'une **Taxonomy Topics**.
- Création d'un **Content Type Article Bis** comportant un champ d'**Entity Reference** pointant vers cette taxonomie.

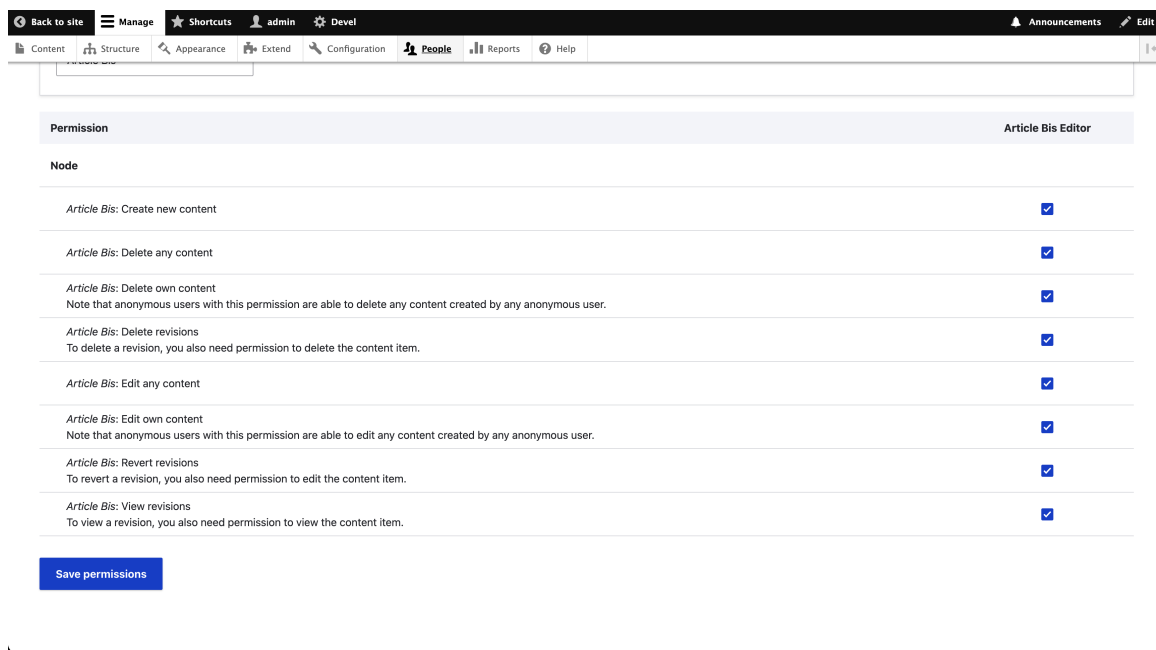


FIGURE 3.1 – Nouveau content type Article Bis

Une fois construit, l'export des configurations YAML s'est fait via Drush :

```
1 drush config:export -y
```

Listing 3.1 – Exportation de la configuration

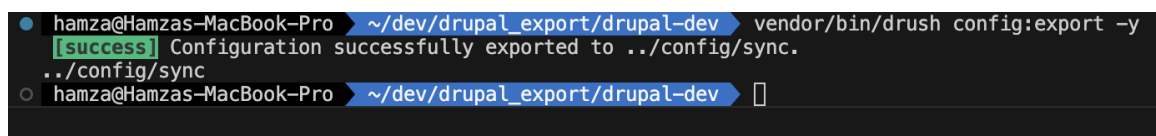


FIGURE 3.2 – Commande drush config :export

Ces fichiers YAML (`system.site.yml`, `node.type.article_bis.yml`, etc.) ont ensuite été copiés dans le dossier `config/sync/` de l'environnement de production.

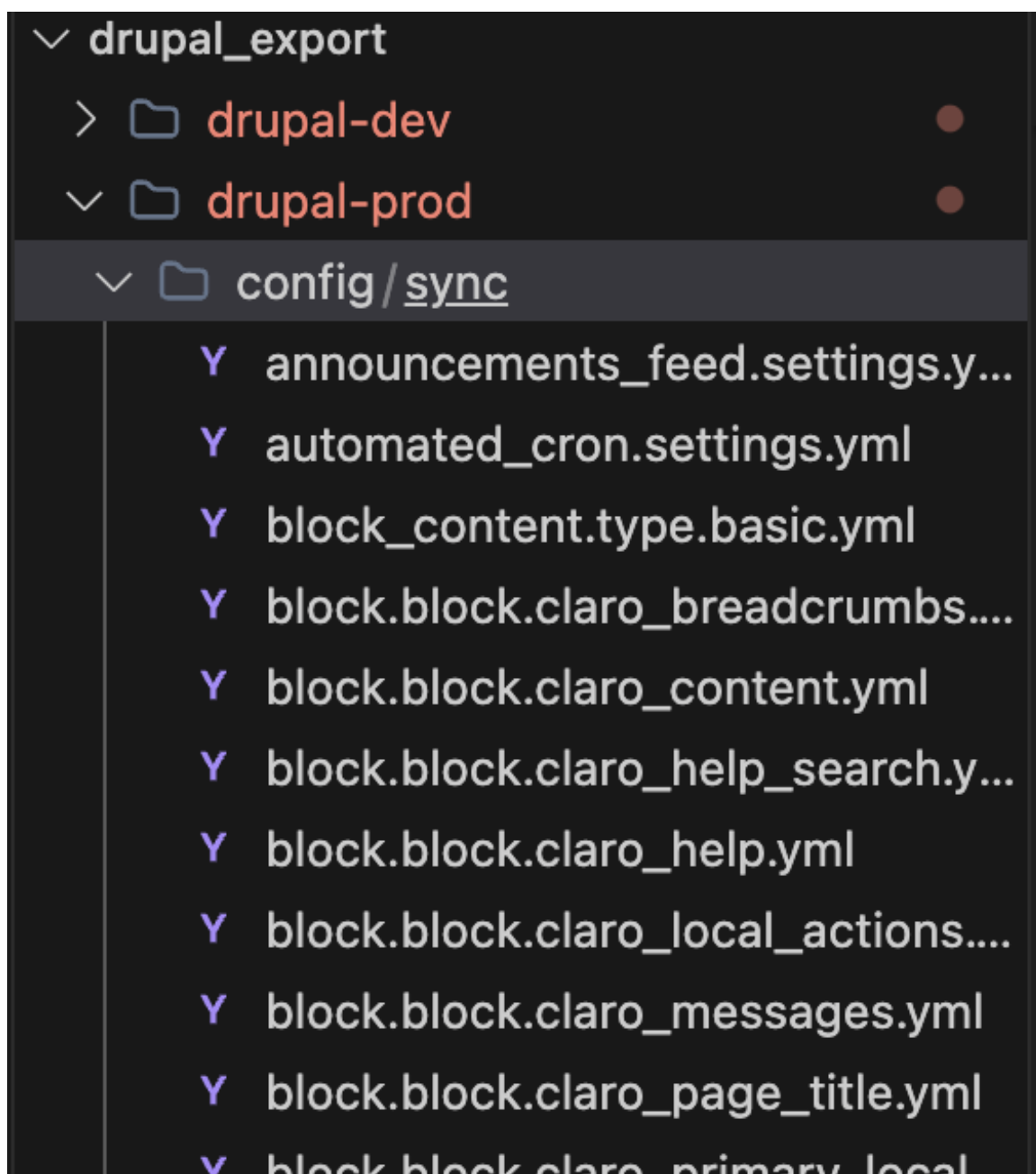


FIGURE 3.3 – Fichiers .yml copiés dans l'environnement de production

3.2.1. Obstacles lors de l'import (Drush config :import)

L'importation de cette configuration sur la production a soulevé trois erreurs classiques révélant les garde-fous de Drupal :

Erreur 1 : Module manquant (Unable to install the devel module)

L'exportation indiquait que `devel` devait être activé. Cependant, ce module n'était pas téléchargé (via Composer) sur la production, car c'est un outil de dev.

Solution : Nous avons installé le module `config_split` sur Dev, qui permet de définir des configurations qui ne s'exportent/s'activent que sur certains environnements (ex : exclure Devel de l'export principal).

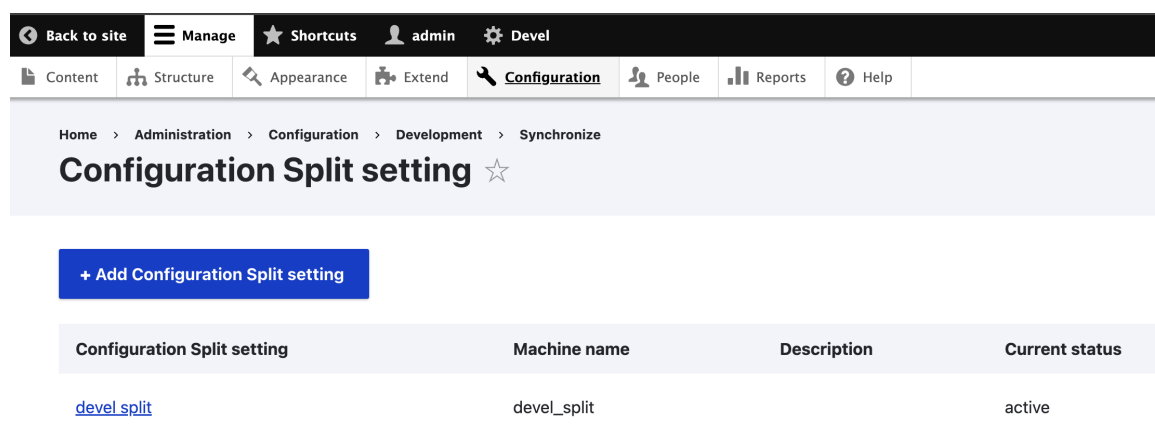


FIGURE 3.4 – Configuration du module Split pour séparer la configuration à exporter

Erreur 2 : UUID du site incompatible (Site UUID does not match)

Chaque installation de Drupal génère un *Site UUID* unique. Drupal bloque l'importation de configuration provenant d'un site avec un UUID différent pour des raisons de sécurité.

3.3. Résolution de l'UUID et Stockages de configuration

La compréhension de la mécanique interne de Drush a été cruciale pour résoudre le problème d'UUID.

3.3.1. Active Storage vs Sync Storage

- **Active Storage (Base de données)** : C'est l'endroit où le site Drupal lit actuellement sa configuration. Les commandes comme `drush cget system.site uuid` piochent ici.
- **Sync Storage (Fichiers YAML)** : C'est le répertoire `config/sync/` qui contient les configurations à importer (nos exports de *dev* lus via `grep uuid`).

3.3.2. Pourquoi `drush cget + cset` a échoué initialement ?

Exécuter `drush cget` sur la production nous a simplement retourné l'UUID actuel de la base de données de production. Le réinjecter via `cset` revenait à dire "remplace mon UUID par mon UUID actuel", ce qui ne résolvait pas le décalage avec l'UUID de développement contenu dans les fichiers d'import.

3.3.3. La solution

Pour forcer le site de production à s'identifier comme un clone du site de développement :

```
1 # 1. Recuperation de l'UUID source directement depuis le systeme de
   dossiers (Sync Storage de DEV) :
2 cat ../drupal-dev/config/sync/system.site.yml | grep uuid
```

```
3
4 # 2. Injection de cette valeur cible dans la base de donnees de PROD
   (Active Storage de PROD) :
5 drush cset system.site uuid "93da7e47-e251-40c4-a63e-63fadc9b6dc6" -
   y
6
7 # 3. Vide le cache pour verouiller le nouvel UUID
8 drush cr
```

Listing 3.2 – Alignement de l'UUID du site

Une fois le `config.split` en place et l'UUID aligné, l'exécution de `drush config:import -y` sur la production a migré la totalité de la structure sans aucun conflit.

3.4. Gestion de Configuration avec Features

Suite à l'expérience de synchronisation avec l'API de **Configuration Management** native, nous avons testé l'approche alternative proposée par le module **features**, historiquement très populaire sous Drupal 7.

3.4.1. Installation et Dépendances

Sur les deux environnements, nous avons dû installer le module via Composer. Une erreur de dépendance s'est présentée car `drupal/features` requiert `drupal/config_update` qui ne dispose que de versions Alpha pour la branche 2.x. La politique de stabilité minimale (`minimum-stability: stable`) de notre `composer.json` a bloqué l'installation.

Solution : Nous avons explicitement requis la version alpha cible en même temps que `features` :

```
1 composer require 'drupal/features:^3.16' 'drupal/config_update:^2.0
   @alpha'
```

Listing 3.3 – Installation avec gestion de stabilité

3.4.2. Création et Déploiement d'une Feature

Sur l'environnement de développement, l'interface utilisateur de Features (`features_ui`) nous a permis de regrouper le type de contenu "Article Bis", la taxonomie, et le rôle associé sous forme de module personnalisé (`my_content_feature`). Cliqué sur "Write", le module a été automatiquement généré dans le dossier des modules custom de Dev.

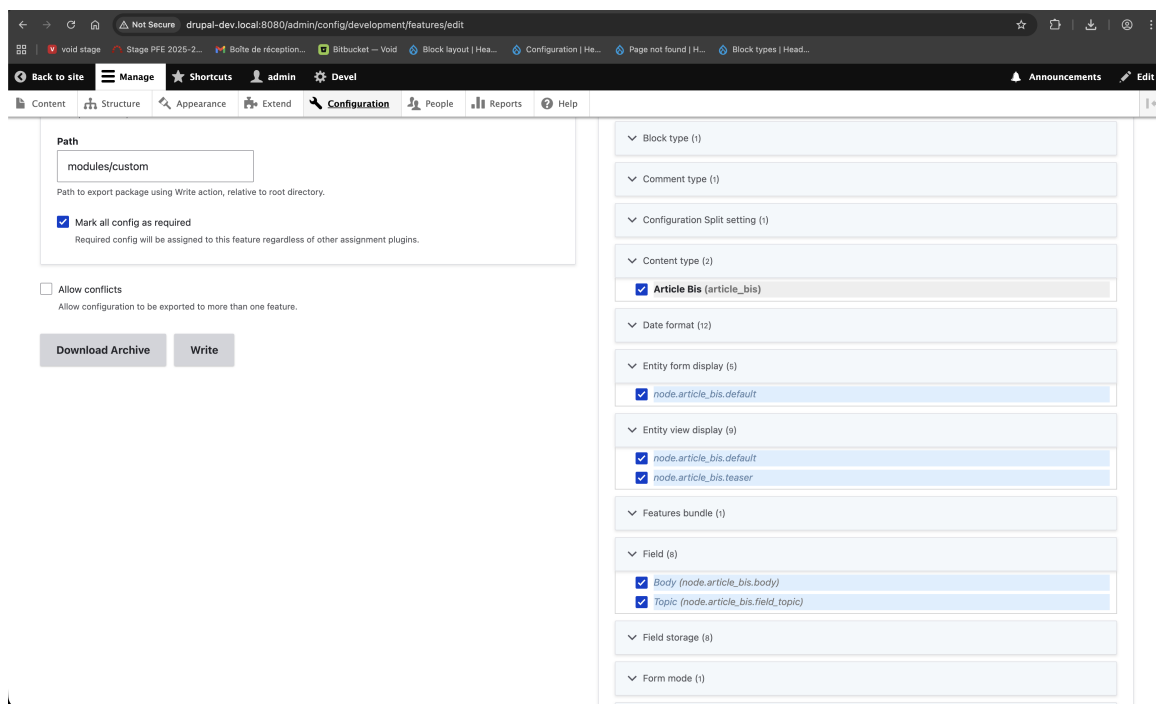


FIGURE 3.5 – feature UI (1)

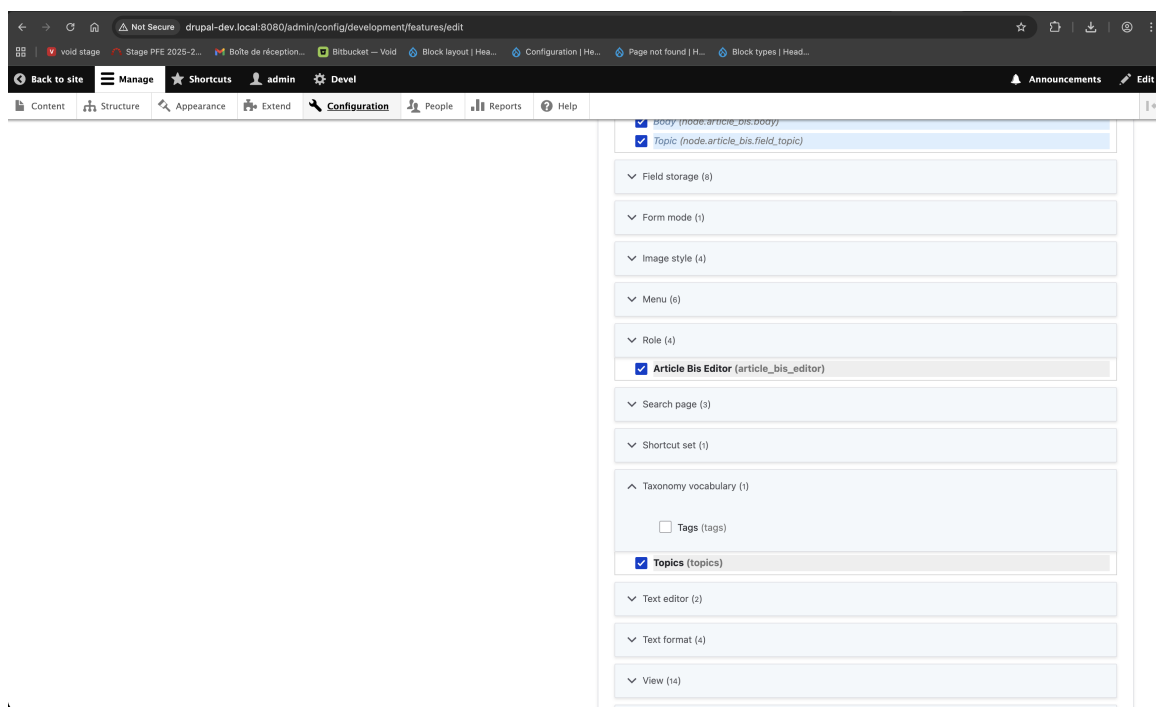


FIGURE 3.6 – feature UI (2)

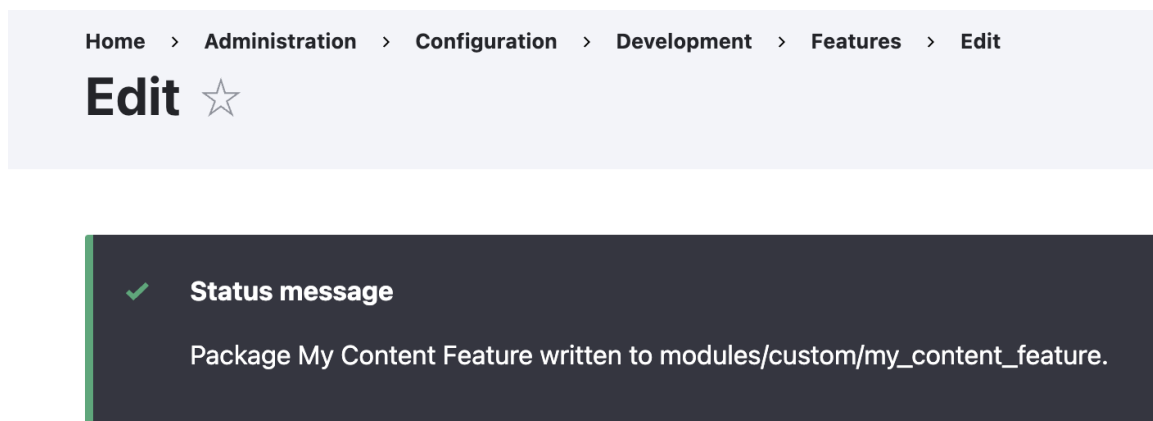


FIGURE 3.7 – feature UI (3) - Succès

Ce dossier a été copié vers l’environnement de production. En production (où `features_ui` n’est pas activé), nous avons utilisé Drush pour activer le module et importer sa configuration :

```
1 drush en my_content_feature -y
2 drush fim my_content_feature -y
```

Listing 3.4 – Importation d’une feature via Drush

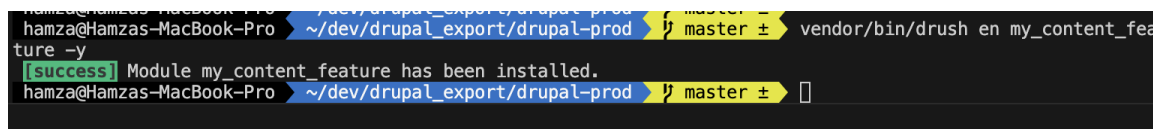


FIGURE 3.8 – Activation et importation de la Feature via Drush

3.4.3. Gestion des Conflits (*Overrides*)

Pour tester la robustesse du système lors de conflits d’états, le scénario suivant a été simulé :

1. Modification directe sur l’environnement Prod : Changement du nom du type de contenu.
2. Modification sur l’environnement Dev : Ajout d’un nouveau champ, puis ré-export du module Feature.
3. Mise à jour sur Prod : Remplacement du dossier module et lancement de `drush fim`.

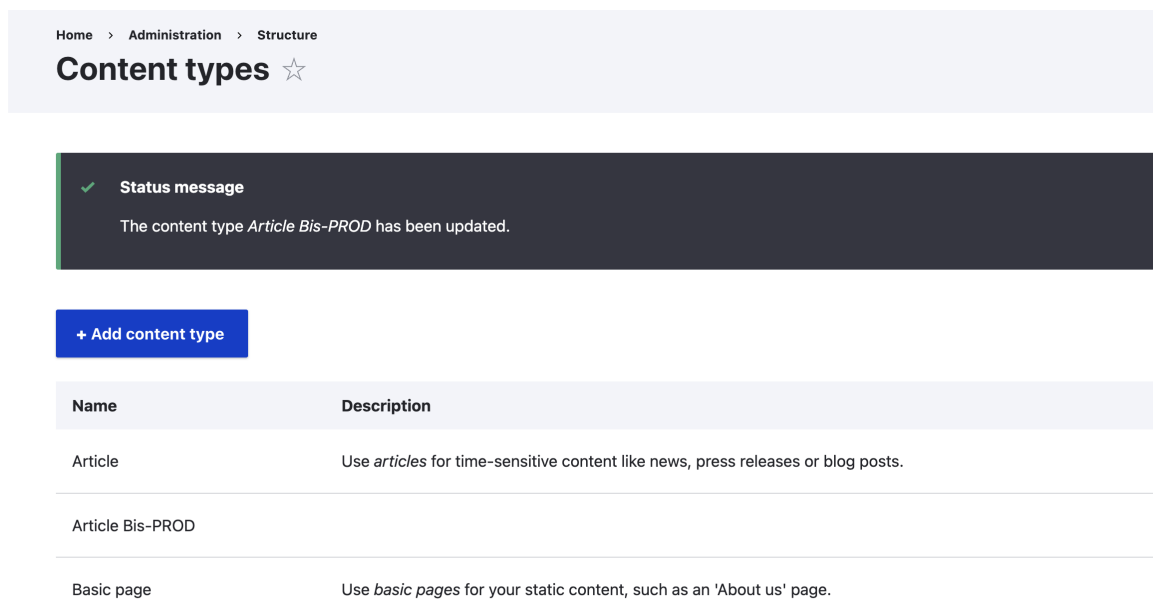


FIGURE 3.9 – Modification du titre du content type Article Bis

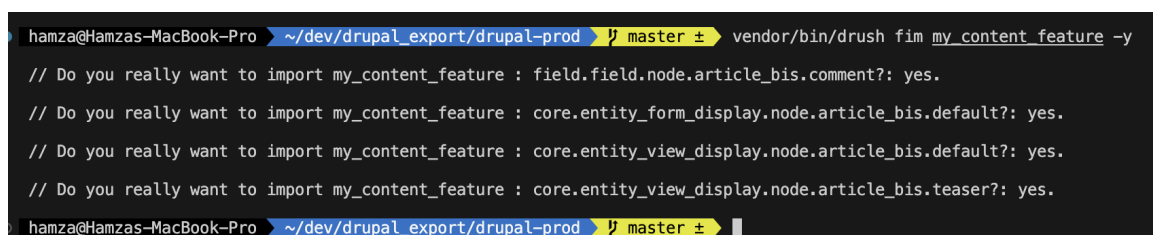


FIGURE 3.10 – Importation de la Feature après les modifications

Résultat observé : L'exécution de l'import `drush fim` a écrasé l'**Active Storage** de production de manière autoritaire. La modification isolée faite sur le titre en production a été perdue au profit de la structure importée de la version dev.

Conclusion : Contrairement au CMI native et `config_split` de Drupal 8+ (qui gèrent des différentiels d'états globaux), **Features** package la configuration comme entité de code monolithique locale qui impose un comportement "All or Nothing" complexe lors de fusions de données actives. C'est pour cela qu'il a cédé sa place comme leader de synchronisation d'environnements dans l'écosystème moderne.

Chapitre 4- Jour 4 : Work with Cache

Ce quatrième jour a été dédié à la maîtrise de l'API de cache de Drupal. Nous avons abordé les concepts de *max-age* (durée de vie maximale d'un cache) et de *cache-tags* (invalidation ciblée basée sur les entités), essentiels pour optimiser les performances des applications sans sacrifier la fraîcheur des données.

4.1. Mise en place d'un API endpoint

Pour expérimenter avec le cache, nous avons créé un module personnalisé `day4_cache` exposant une route API `/api/articles`.

4.1.1. Création de la route

La route a été définie dans le fichier `day4_cache.routing.yml` pour diriger vers un contrôleur spécifique.

```
1 day4_cache.api_articles:
2   path: "/api/articles"
3   defaults:
4     _controller: '\Drupal\day4_cache\Controller\
5                   ArticleCacheController::getArticles'
6   requirements:
7     _access: "TRUE"
```

Listing 4.1 – Définition de la route API

4.2. Le Cache basé sur le Temps (Max-Age)

Dans un premier temps, nous avons implémenté le cache basé sur le temps en utilisant *max-age*. Le contrôleur chargeait 3 articles codés en dur via un `entityQuery` et retournait leur ID et titre au format JSON.

```
1 $response = new CacheableJsonResponse($data);
2
3 // Configuration du max-age (ex: mise en cache pour 60 secondes)
4 $cache_metadata = new CacheableMetadata();
5 $cache_metadata->setCacheMaxAge(60);
6
7 $response->addCacheableDependency($cache_metadata);
8 return $response;
```

Listing 4.2 – Implémentation du Max-Age dans le contrôleur

Test de fonctionnement : Après avoir modifié le titre d'un des trois articles dans le back-office, la requête API retournait toujours l'ancien titre. Les données ne se rafraîchissaient pas instantanément car la réponse restait verrouillée dans le cache jusqu'à l'expiration du délai de 60 secondes.

4.3. Le Cache basé sur les Entités (Cache-Tags)

Pour remédier au délai de rafraîchissement imposé par `max-age`, nous sommes passés aux *cache-tags*. Les entités Drupal (comme les noeuds) déclarent automatiquement des tags (ex : `node:1`, `node:2`).

En liant notre réponse JSON à ces entités spécifiques, Drupal s'occupe d'invalider le cache de notre API à l'instant même où l'un de ces noeuds est sauvegardé ou modifié.

```

1  $cache_metadata = new CacheableMetadata();
2
3  foreach ($nodes as $node) {
4      if ($node->access('view')) {
5          $data[] = [
6              'nid' => (int)$node->id(),
7              'title' => $node->getTitle(),
8          ];
9          // Fusion des cache tags de ce noeud spécifique
10         $cache_metadata->addCacheableDependency($node);
11     }
12 }
13
14 $response = new CacheableJsonResponse($data);
15 $response->addCacheableDependency($cache_metadata);
16
17 return $response;

```

Listing 4.3 – Implémentation des Cache-Tags

Test de fonctionnement : Avec l'utilisation de la dépendance de cache liée aux objets `$node`, la modification d'un titre depuis le back-office a provoqué l'invalidation automatique du cache. Le rafraîchissement des données lors du prochain appel à `/api/articles` est devenu instantané, sans avoir à attendre un délai d'expiration.

4.4. Inspection des Cache Tags

Afin d'auditer ces comportements, il était nécessaire de rendre visible les en-têtes de cache HTTP de Drupal.

Nous avons activé le fichier `settings.local.php` et vérifié la présence du paramètre de débogage dans `development.services.yml` :

```

1  parameters:
2      http.response.debug_cacheability_headers: true

```

Listing 4.4 – Activation des en-têtes de débogage du cache

Une fois les caches reconstruits, l'inspection des requêtes réseaux dans le navigateur affichait clairement les *Response Headers* :

— X-Drupal-Cache-Tags: config:user.role.anonymous http_response node:1 node:2 node:3 node_view

— X-Drupal-Cache-Max-Age: -1

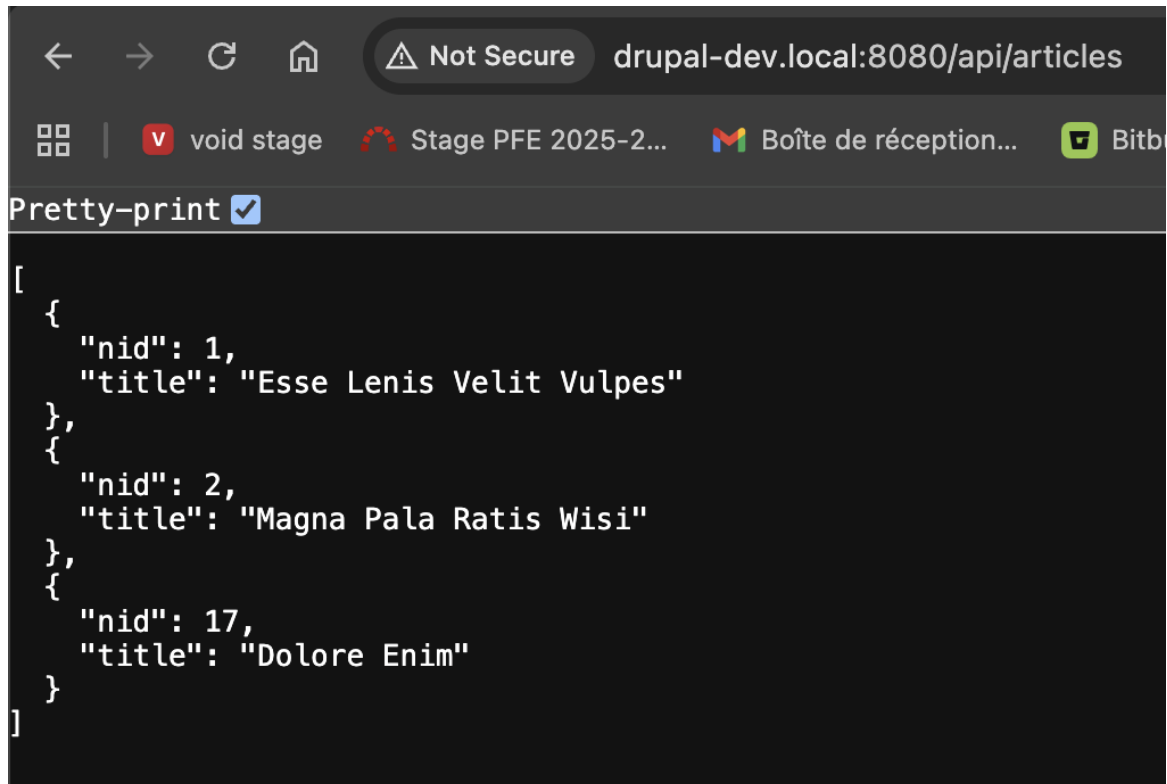


FIGURE 4.1 – Réponse de l'endpoint /api/articles

Name	×	Headers	Preview	Response	Initiator	Timing	Cookies
articles		▼ Response headers	☐ Raw				
		Cache-Control		must-revalidate, no-cache, private			
		Connection		Keep-Alive			
		Content-Language		en			
		Content-Type		application/json			
		Date		Thu, 12 Mar 2026 13:28:46 GMT			
		Expires		Sun, 19 Nov 1978 05:00:00 GMT			
		Keep-Alive		timeout=5, max=99			
		Server		Apache/2.4.66 (Unix)			
		Transfer-Encoding		chunked			
		X-Content-Type-Options		nosniff			
		X-Drupal-Cache		UNCACHEABLE (request policy)			
		X-Drupal-Cache-Contexts					
		X-Drupal-Cache-Max-Age		-1 (Permanent)			
		X-Drupal-Cache-Tags		http_response node:1 node:17 node:2			
		X-Drupal-Dynamic-Cache		HIT			
		X-Frame-Options		SAMEORIGIN			
		X-Generator		Drupal 11 (https://www.drupal.org)			
		X-Powered-By		PHP/8.3.30			

FIGURE 4.2 – Inspection des en-têtes de réponse X-Drupal-Cache depuis le navigateur

| Chapitre 5- Jour 5 : Work with Data Migration

Ce cinquième jour a été consacré à l'exploration de l'API de Migration de Drupal (*Migrate API*), un outil puissant et extensible permettant d'importer des volumes importants de données depuis de multiples sources externes vers des entités Drupal natives.

Dans le cadre de cette session, nous avons analysé l'architecture du système de migration de Drupal à travers la résolution de quatre problématiques fondamentales.

5.1. Le rôle du plugin `migration_lookup`

Le **Process Plugin** `migration_lookup` sert à relier et cartographier les relations entre des entités qui ont été importées à travers des processus de migration distincts.

Lorsqu'une migration de données est effectuée (par exemple : importation des *Auteurs* lors d'une Migration A, puis des *Articles* lors d'une Migration B), l'API Migrate de Drupal conserve en interne une table de correspondance (*mapping table*). Cette table relie les identifiants originaux de la source aux nouveaux identifiants générés par Drupal.

Ainsi, lors de la migration des articles (Migration B), on utilise `migration_lookup` sur le champ référençant l'auteur pour indiquer au système : « *Consulte la table de correspondance générée par la migration des Auteurs, prends l'identifiant original de l'Auteur fourni par ma source, et attribue à cet Article le nouvel identifiant Utilisateur Drupal qui lui correspond.* »

5.2. Importer depuis une base de données MySQL

L'API Migrate de Drupal repose intrinsèquement sur des **Plugins de Source** (*Source Plugins*). Si la source des données n'est pas un fichier CSV, l'adaptation s'opère simplement par le changement du plugin défini dans le fichier YAML de la migration.

Pour migrer des données depuis une base de données MySQL externe, la démarche s'articule ainsi :

1. **Définition de la connexion à la base de données** dans le fichier `settings.php` de l'environnement, en utilisant une clé de tableau spécifique (par exemple, `'migrate_db'`).
2. Dans le fichier YAML de la migration, au sein de la section **source**, **modification du plugin** vers `sql`. Cette action requiert soit l'écriture d'un **Source Plugin** personnalisé héritant de `SqlBase` pour y définir la requête SQL précise, soit l'utilisation d'un module contribué fournissant des plugins SQL génériques.
3. Ajout de la clé de base de données définie (**key**: `'migrate_db'`) dans la configuration du plugin de source, afin que celui-ci sache exactement quelle base de données interroger pour extraire les enregistrements.

5.3. Annulation (Rollback) d'une migration

Le système offre la possibilité d'annuler complètement et facilement une migration spécifique grâce à **Drush**.

Étant donné que l'API Migrate conserve une trace de chaque élément importé avec succès au sein d'une table de correspondance dédiée (`migrate_map_MON_ID_DE_MIGRATION`), le système sait avec une certitude absolue quelles entités ont été créées par cette migration (et lui appartiennent donc) et lesquelles ne le sont pas.

Afin d'exécuter un retour en arrière (*rollback*) et de supprimer **uniquement** les entités créées par une migration donnée, il suffit d'exécuter la commande suivante dans le terminal :

```
1 drush migrate:rollback MON_ID_DE_MIGRATION
2 # ou sa version raccourcie :
3 drush mr MON_ID_DE_MIGRATION
```

Listing 5.1 – Commande Drush pour annuler une migration

5.4. Traitement des données de champ (Pipeline)

L'API Migrate de Drupal utilise un concept de « **Pipeline de Processus** » (*Process Pipeline*) qui permet de transformer, nettoyer et altérer les données à la volée, juste avant qu'elles ne soient sauvegardées dans l'entité de destination.

Pour modifier le format d'une chaîne de caractères (comme par exemple en réduire la longueur à 10 caractères), le système s'appuie sur des **Process Plugins**.

Dans ce cas de figure précis, la solution est d'utiliser le plugin de base fourni par le cœur (*core plugin*) nommé **substr**, directement au niveau du fichier de configuration YAML :

```
1 process:
2   field_short_description:
3     plugin: substr
4     source: my_very_long_source_string
5     start: 0
6     length: 10
```

Listing 5.2 – Utilisation du plugin substr dans le processus de migration

Note : Dans la situation où la transformation requise s'avère extrêmement complexe ou profondément liée à la logique métier du projet, empêchant l'utilisation des plugins natifs, il est aisé de concevoir un **Process Plugin** personnalisé en PHP. Cela s'effectue en créant une nouvelle classe étendant `ProcessPluginBase` et en encapsulant toute la logique de transformation au sein de sa méthode `transform()`.